Mechanising Count-One Ownership for Array Updates in Lean 4

Gavin Mendel-Gleason
Scidonia Limited

Lean statements are quoted from the development; hypotheses are abbreviated where the text says so.

Abstract

A reference-counted array update sits on a complexity cliff: a write to a uniquely referenced, non-persistent array mutates it in place, while a write to a shared array copies the whole array, so a sequence of updates can cost linear or quadratic time according to aliasing alone. Lean 4 decides the branch at run time from the heap's count and persistence, and its source type system has no ownership or linear types. A static discipline would make the in-place branch a compile-time guarantee; adding one to Lean would be a language change. Can the guarantee be recovered without one? We show that it can, for a recognised fragment of Lean's compiler IR, by an external count-one discipline proved in Lean 4 against model semantics. The proof rests on a complete accounting of owning roots and incoming heap edges; on preservation for a restricted flat fragment, whose accepted execution records only exclusive writes; and on a counted extraction, call and restoration protocol proved at its accounted boundaries, since the temporary pair that the emitted code allocates makes the accounting false between them. A recogniser ties the emitted LCNF to a model program; the flat composed guarantee relates the recognised trace, the model run and a runtime model at the designated write and takes no accounting premise. A nested theorem composes the finite-path correspondence, the counted chain and the counted interpreter's recognised-prefix boundary at one level of a supported path, conditional on an explicit, stronger counted child-call contract, with a closed kernel-checked instance. A second theorem runs the counted interpreter over the whole recognised fragment—extraction, child call, restoration and return—and relates its outcome to the model's run of the same term by heap compatibility (the same cell at every name, the model cursor no higher than the counted one), with the same result, fault and write trace. The claim is a checked count one for the fragment, not a physical count, a step-by-step agreement inside the pair-live window, or C verification.

2012 ACM Subject Classification Theory of computation → Logic and verification; Software and its engineering → Software verification and validation

Keywords and phrases reference counting, ownership, compiler verification, Lean 4, separation logic

Digital Object Identifier 10.4230/LIPIcs...

1 Introduction

An array update should cost the same whoever else holds the array, but in a reference-counted language it does not. Lean 4's arrays are reference counted: a write mutates in place when the array is uniquely referenced and non-persistent, and otherwise copies it. `lean_is_exclusive` makes this choice at run time [3, 9, 10], from heap state the source does not mention, so n updates over an n-element array can cost linear or quadratic time according to sharing alone. A static ownership discipline would make the in-place branch a compile-time guarantee; Lean 4 has no ownership or linear types, and adding them is a language change.

Can the guarantee be recovered without one? This paper argues yes, for a recognised fragment of Lean's compiler IR. We define a count-one discipline for a fragment recognised in the emitted LCNF and prove in Lean 4 that its counted part preserves a complete ownership accounting, that the counted extraction and restoration protocol preserves it at its boundaries, and that the recognised code corresponds to a model program. The guarantee stops short


© Gavin Mendel-Gleason;
licensed under Creative Commons License CC-BY 4.0


of physical counts and C semantics. The novelty is the proved connection, not a new type system.

The first obstruction is that count one is not one owner. The natural invariant equates the heap's stored count with the context's multiplicity, but a multiplicity is not an owner count: binding a second owned variable preserves the invariant without changing the multiplicity, and a kernel-checked countermodel gives two owning variables on a count-one cell. A count of one can therefore hide an alias.

The repair is complete accounting: a duplicate-free, sound, complete and bound support of owning variables, a live-cell domain, and $rc(r) = \mathit{IncomingCount}(r) + \mathit{EnvOwnerCount}(r)$, the heap fields that reference r plus the owning variables that denote it. At count one the equation forces one owning root and no incoming edge, or no owning root and exactly one incoming edge; two owning variables are the same variable. The integer now has a meaning.

Preservation is then a theorem about a restricted fragment: no increments, only single decrements, and the fresh-binding certificates the checker cannot see. From an accounted state it runs without a fault, records only exclusive writes, and satisfies the accounting at every state it reaches, at the final support the step arithmetic determines. One premise is supplied—what the written slot holds, which no context records; it is discharged at the initializer's measured write, not in general. The checker itself is unchanged.

The second obstruction is the emitted protocol. A nested update takes the element out of the array, allocates a pair holding the parent and child, projects both, frees the unique pair, calls on the child, and puts the result back. After the take-out the pair's field points at the parent while the caller's variable still owns it: the equation reads one = one plus one, and the projections raise both counts. The accounting relation is therefore a boundary relation, attained at entry, after the release and after restoration. We prove the pair's traffic neutral at release, keep the no-free and retained-alias controls that leave counts elevated, derive restoration's mutate branch from the protocol and child-call boundaries, and compose take-out, an admitted child call and restoration into one chain, related to the model by a compatibility relation that tolerates the allocator's cursor.

The third obstruction is the compiler connection, since Lean's compiler chooses the emitted shape, inlines and renames. The checker's programs are explicit syntax over a model heap, so the development is a deep embedding; a recogniser walks the actual LCNF value, maps a recognised prefix to one of those programs, pins operation order, arities, operand identities and branch constructors, and rejects token-preserving near misses. The trace is executed and compared with the model run. For the flat write this yields a composed guarantee (`principal_write`) over the executed prefix, the model run and a runtime model at the designated write, with no accounting premise. No universal rejection theorem is claimed.

The nested result is narrower. The finite-path correspondence relates the executed nested prefix to the model run, and the ordinary child contract is derived at every level from one supplied leaf. The counted statement composes that correspondence with the counted chain and the counted interpreter's recognised-prefix boundary at one level of a finite supported path, from an accounted caller state. Its child call is a stronger contract: it requires the caller's complete accounting at the body's return, which the finite-path induction does not derive. A closed kernel-checked instance inhabits the statement at the recogniser's layout. The counted interpreter's run over the whole *p.fragment* (`principal_counted_fragment`) is `NestedOutcome.Compat` to the model's run: heap compatibility with the same result, fault and write trace. The boundary controls remain. Neither a step-by-step agreement inside the pair-live window nor the emitted callee is claimed.

The closest predecessor, Huisinga's static uniqueness analysis for a Lean IR model [4], has no compiler connection and leaves soundness as future work. Perceus and Counting Immutable Beans [9, 10] insert counts and reuse as a compiler transformation; here the question is when count one is justified, over an existing IR. Linear Haskell [2] and Clean's uniqueness types [1] make ownership or uniqueness a source-level type property, while `Array.markLinear` [6] establishes it dynamically.

The guarantee is relative to entry and agreement facts, recogniser and layout facts, one supplied leaf contract, the stronger counted child-call contract, the primitive contracts, and the heap-to-runtime-model correspondence. The emitted callee is not captured; extraction and restoration are not trace events although their targets are proved exclusive; and C semantics, replay and address identity, whole-path non-growth and the source correspondence remain open. "In place" means the modelled operation neither copies nor allocates; the pair still allocates, and neither count is physical.

Three contributions.

- **Complete accounting and preservation.** A relation enumerating owning variables, covering the live cells, and equating each count with its incoming edges plus its owning roots yields the count-one dichotomy and rejects the two-owner countermodel. The preservation theorem `supported_run` takes an accounted state and one supplied slot fact and concludes a fault-free run, exclusively flagged writes, and accounting at every state reached.
- **A counted protocol at its accounted boundaries.** The pair's traffic is neutral at release; the no-free and retained-alias controls leave counts elevated; restoration's mutate branch is derived from the protocol and child-call boundaries; and `counted_nested_chain` composes take-out, an admitted child call and restoration, related to the model by a cursor-tolerant compatibility relation. The caller's accounting at the child's return is an explicit stronger contract.
- **A checked connection from emitted LCNF to the models.** A recogniser walks the actual declaration, pinning operation order, arities, operand identities and branch constructors; its trace is executed and related to the model. The flat composed guarantee relates recognised execution, model run and runtime model at the designated in-place write, with no accounting premise. The nested statement (`principal_supported_path`) composes finite-path correspondence, counted chain and counted-prefix boundary at one supported level, from an accounted caller state and the counted child-call contract, with a closed kernel-checked instance.

2 Setting and the running example

Lean 4 compiles through a sequence of IRs. The last LCNF phase inserts reference-count operations, and the backend emits C. The runtime decides an array update with `lean_is_exclusive`: count one and non-persistent means in place; every other case copies. In this setting the object is single-threaded; count one and non-persistence characterise this test and are not a claim about every Lean runtime mode. The development pins the toolchain, the sibling revision and the captured LCNF artifacts, so every claim names the code it is about.

The problem is not to implement reference counting, but to know statically that the count at a particular write is one. Lean's source type system has no linearity or uniqueness; adding one would be a language change. We therefore state the property over the compiler's code and prove the connection, rather than extend the type system.

2.1 Model programs

The checker does not run on Lean source or on LCNF directly. Its programs are two small inductive types, interpreted over the model heap and environment; the compiler connection is a mapping from the recognised LCNF to these values.

A `LetF` is one of `lit n`, `size a`, `isShared x` or `checkAlias x`.

```
inductive Code where
  | let (x : Var) (v : LetF) (k : Code)
  | inc (x : Var) (n : Nat) (k : Code)
  | dec (x : Var) (n : Nat) (k : Code)
  | check (x : Var) (k : Code)
  | setArr (x : Var) (i : Var) (v : Var) (k : Code)
  | ret (x : Var)

inductive NCode where
  | base (c : Code)
  | takeOut (a i fst snd : Var) (e : Ref) (k : NCode)
  | putBack (a e i result : Var) (k : NCode)
  | callAt (state path x result : Var) (body : NCode) (k : NCode)
  | ret (x : Var)
```

A `Code` is the straight-line array-write fragment; an `NCode` adds extraction, restoration and the recursive call. Variables are `Nat`, references are `Nat`, `Val` is a reference, a scalar or erased, and a `Heap` maps references to objects with a field array, a count and a persistence flag. `run`/`runN` execute these values and `check`/`checkN` are the checkers; the soundness theorems are about this semantics. They are neither Lean source nor LCNF: `projectPrefix` and `projectNested` walk an LCNF value, `Prefix.toModel` and `indexModelTerm` read a `Code`/`NCode` off the recognised operands, and `execAST`/`execNestedAST` execute the recognised trace. The correspondence theorems relate that execution to the model run.

2.2 The nested write

The source is the array case of `Layout5.setAt` in the sibling. It takes the element out, checks the extracted focus, recurses on it and puts the result back:

```
| _, .index i q, y, s =>
    if h : i < s.size then
      match takeOutAt (assertExclusiveArray openSite s) i h (stateDefault _) with
      | ⟨(sib, focus), hbound⟩ =>
        plugAt sib i hbound (setAt q y (assertExclusive structSite focus))
    else s
```

The emitted C for this arm is the sequence the development names: a `lean_is_exclusive` test for the array, the `takeOut` exchange (`lean_array_fget`, `lean_array_fset`, the pair allocation), the child check on the extracted element, the recursive call and the `lean_array_fset` put-back. The recogniser `projectNested` accepts the `Path.index` arm, `projectPair` accepts the two pair arms, and `execNestedAST` executes the accepted trace. The model term is `Path.build path value body ls`; `checkN` checks it from `initCtx`.

The example separates what the tool computes from what a client supplies. The compiler computes the LCNF and the C; `projectNested` computes the prefix; `Prefix.toModel`/`indexModelTerm` compute the model program; `checkN` computes the checker result. The client supplies the source declaration, the `Path` and the take-out name, and the proofs of `Agrees`, `LevelMatch`, `PathContract` and `EntryInv` (the path/slot and entry facts the

checker cannot see). The recursive call is a contract on an admitted model body: the ordinary child contract is derived at every level by the finite-path derivation, with one leaf computation supplied, while the counted call takes the caller's accounting at the body's return as an explicit premise that the derivation does not construct. The depth-two witness instance in `OwnershipTest/PathRecursion.lean` takes model index 1 at both levels and leaf `.base (.setArr 9 5 2 (.ret 9))`; its tip slot 1 goes from scalar 6 to scalar 1, the root and child slot 1 are restored to references 1 and 3, and the root's sibling slot 0 keeps scalar 5. A deeper level that takes its index by re-projecting the continuation is outside the model, because the flat environment cannot express the emitted per-invocation index; the emitted call's meaning (N10) is not captured. `OwnershipTest/PathConsolidation.lean` supplies a depth-three leaf in `takeOut`/`putBack` form as a supplied-leaf witness, not an instance of the correspondence.

3 The accounting invariant

The model heap, its counts and exclusivity are the semantic side of the judgement: `Inv` ties the context's multiplicity to the heap count, and `Exclusive` is count one and non-persistence.

▷ Claim 1 (The raw invariant and the exclusivity predicate).

```
structure Inv (H : Heap) (env : Env) (c : Ctx) : Prop where
  rc_eq         : ∀ r, H.rc r = c.mult r
  owned         : ∀ x r, c.us x = .owned → c.refOf x = some r →
                    env x = some (.ref r) ∧ 1 ≤ c.mult r
  borrow        : ∀ x r, c.us x = .borrow → c.refOf x = some r →
                    env x = some (.ref r) ∧ H.cells r ≠ none
  scalar        : ∀ x, c.us x = .scalar → ∃ n, env x = some (.scalar n)
  nonPersistent : ∀ r o, H.cells r = some o → o.persistent = false
```

`Exclusive H r` means that `r` is a live, non-persistent cell whose count is one.

The displayed invariant relates the heap count to the context multiplicity, but it does not count the environment's owning variables: `Inv.owned` only requires `1 ≤ c.mult r`. `bindOwned` preserves `Inv` without changing `mult`, so two owned variables can denote one count-one cell. The raw invariant therefore does not imply physical uniqueness. The kernel countermodel `alias_not_accounted` constructs exactly that state: `initCtx` has one owner, and binding a second owned variable leaves the multiplicity at one. The initializer's context does satisfy `OwnersAccounted` (`initCtx_accounted`). `CountFidelity` is not a consequence of `Inv`, but it is a consequence of the relation below (`countFidelity`); the residual runtime fidelity is not proved.

▷ Claim 2 (Complete accounting).

```
structure Accounted (support : List Var) (H : Heap) (env : Env) (c : Ctx) : Prop
    where
  inv              : Inv H env c
  support_nodup    : support.Nodup
  support_sound    : ∀ x, x ∈ support → c.us x = .owned
  support_complete : ∀ x, c.us x = .owned → x ∈ support
  support_bound    : ∀ x, x ∈ support → ∃ r, c.refOf x = some r
  domain           : ∀ r o, H.cells r = some o → r < H.next
  count            : ∀ r, H.rc r = IncomingCount H r + EnvOwnerCount c support r
```

The relation is deliberately redundant: the support is a duplicate-free enumeration of every owning variable, each entry is bound to a reference, every live cell lies inside the range the edge count enumerates, and the count equation holds over two independently computed quantities.

```
def IncomingCount (H : Heap) (r : Ref) : Nat :=
  ((List.range H.next).map fun q =>
    match H.cells q with
    | some o => (o.fields.toList.filter (fun v => v = .ref r)).length
    | none => 0).sum

def EnvOwnerCount (c : Ctx) (vars : List Var) (r : Ref) : Nat :=
  (vars.filter (fun x => c.us x = .owned ∧ c.refOf x = some r)).length

def CountFidelity (H : Heap) (c : Ctx) (vars : List Var) : Prop :=
  ∀ r, H.rc r = IncomingCount H r + EnvOwnerCount c vars r
```

`mult_eq` reads the equation through `Inv.rc_eq`; `count_one_dichotomy` splits count one into one owning root with no incoming edge, or no owner with exactly one incoming edge; `at_most_one_owner` says two owning variables at count one are the same variable; `countFidelity` is the equation in the retained predicate's shape; and `ownersAccounted` recovers the environment-side predicate. The initializer establishes the relation (`initCtx`).

The supported fragment.

`Supported c s` is indexed at the pre-state `c`; each constructor states the checker's own rule at `c` and its continuation at the checker's post-state, and carries the certificates the checker cannot see: a fresh alias target and a fresh scalar-`.let` target. The fragment has eight constructors; `.inc` is absent and every `.dec` is the single-count form `.dec x 1`, where `decOne`'s premise is exactly the checker's `n < c.mult r` at `n = 1`. From `Supported c s` the checker accepts the program (`supported_check`), which is what lets a preservation theorem follow a fault-free run. The checker itself is unchanged.

The store's slot fact is deliberately absent from the constructors: no context records which value a cell's selected slot holds, so it is the `StoreSlots` premise of `supported_run` rather than a constructor premise. `StoreSlots H env s` is a recursive predicate over the supported program, supplied from the heap and environment alone (it may not assume the invariant): at the store the run reaches, the array cell's selected slot holds a scalar, so the in-place write replaces a value rather than an edge. `storeSlots_init` discharges it for the initializer's measured write; it is not a general `StoreSlots` for `p.toModel`.

4 The checker and preservation

4.1 The judgement and its rules

A context records, for each variable, whether it is owned, borrowed, scalar or dead, which reference it denotes, and the multiplicity of that reference. The checker is a total function from a context and a program to a successor context or failure; the frame stack records open extractions. The rules in Table 1 are ownership obligations: a write is admitted only for an owned handle of multiplicity one, an extraction only with the frame discipline, and a call only with an isolated body that returns the state it was handed.

The object language is untyped: there is no type former for a program, and the checker does not infer one by unification. The typing judgement is $c \vdash \mathit{code} \Rightarrow c'$, defined to hold

exactly when `check c code = some c'`; the context is the type of the program point. A context is a usage, reference and multiplicity map; the checker is syntax-directed.

```
inductive U where
  | owned
  | borrow
  | scalar
  | dead

structure Ctx where
  us    : Var → U
  refOf : Var → Option Ref
  mult  : Ref → Nat
```

A frame records the array, index and element of an open extraction; `check : Ctx → Code → Option Ctx` and `checkNF : Ctx → List Frame → NCode → Option Ctx` are total.

The judgement is syntax-directed: each constructor has one rule, the context is an input, and the successor context is computed by `setUs`, `move`, `bindOwned`, `inc` and `dec`. There are no metavariables, no unification and no implicit type arguments in the object language; the only computation is the context update. This is why the discipline makes the count visible at the type level: at the write, the array's handle has usage `owned` and multiplicity one in the context, and the rule refuses every other usage.

Rule	Obligation	Effect
`let x (lit n)`	—	`x` scalar
`let x (size a)`, `let x (isShared a)`	`a` owned or borrow	`x` scalar
`let x (checkAlias a)`	`a` owned, multiplicity one	`x` is the checked alias; `a` dead
`check x`	`x` owned, multiplicity one	unchanged
`setArr x i v`	`x` owned, multiplicity one; `i`, `v` scalar	exclusive write through `x`
`inc x n`	`x` owned	multiplicity $+n$
`dec x n`	`x` owned; $n <$ multiplicity	multiplicity $-n$; `x` dead
`takeOut a i fst snd e`	`a` owned, multiplicity one; `i` scalar; `e` multiplicity one; `fst`, `snd` dead and distinct; `e` $\neq$ `a`; no open frame at `a`	push the frame; bind parent and element
`putBack a e i result`	`a`, `e` owned, multiplicity one; `i` scalar; `e` $\neq$ `a`; `e` $\neq$ `result`; the frame is `(a,i,e)`; `result` dead	pop the frame; bind the result
`callAt state path x result body`	`state` owned, multiplicity one; `path`, `x` scalar; `result` dead; `CountFree body`; no open frame at `state`; the body is accepted from the isolated context and returns `state`	call the body; bind the result
`ret x`	no frame: `x` owned or scalar; one frame: `x` is the frame's element; deeper: refused	exit

Table 1 The checker's ownership obligations.

Two consequences are worth stating. `dec` is affine, not the last drop: its premise is $n <$ the object's multiplicity, so it consumes a duplicate handle and leaves the object live. And linearity at the write is the side condition `multOf x = 1`, not the usage tag alone.

A derivation.

The first example is the two rules applied in sequence. The premise of the second rule is the context the first produces; no type variable is solved.

$$\frac{\text{us } 0 = \text{owned} \qquad \text{multOf } 0 = 1 \qquad c_1 = c_0.\text{move } 0\ 3}{c_0 \vdash \texttt{checkAlias}\ 0 \Rightarrow c_1} \qquad \frac{\text{us } 3 = \text{owned} \qquad \text{multOf } 3 = 1 \qquad \text{us } 1 = \text{us } 2 = \text{scalar} \qquad c_1 \vdash \texttt{ret}\ 3 \Rightarrow c_1}{c_1 \vdash \texttt{setArr}\ 3\ 1\ 2;\ \texttt{ret}\ 3 \Rightarrow c_1}$$

Composing the two gives $c_0 \vdash \texttt{execSet}\ 0\ 3\ 1\ 2 \Rightarrow c_1$, i.e. `check (initCtx 0) (execSet 0 3 1 2) = some ((initCtx 0).move 0 3)`. The checker computes c_1; the derivation is the rule applications.

The frame stack is the local form of the ownership invariant. An extraction opens a frame naming the array, index and element; a put-back closes the matching frame; a call checks its body under an isolated context and continues with the result.

4.2 Two examples

▶ **Example 3** (A unique write).

```
def execSet (x y i v : Var) : Code :=
  .let y (.checkAlias x) (.setArr y i v (.ret y))

#guard (check (initCtx 0) (execSet 0 3 1 2)).isSome
```

▶ **Example 4** (An aliased write is refused).

```
def aliasSet (x y i v : Var) : Code :=
  .inc x 1 (execSet x y i v)

#guard (check (initCtx 0) (aliasSet 0 3 1 2)).isNone
```

The alias increments the handle, so the write's multiplicity-one obligation fails; the same refusal appears at the context level for a heap whose count is two. This is the static counterpart of the runtime copy: the checker rejects the program before it can copy.

4.3 The invariant and soundness

The entry invariant adds the facts a local checker cannot see: the invariant tree, client separation and root completeness. For fresh arrays `initEntryInv` establishes it, and `checkN_sound_of_entryInv` combines it with the checker.

`check_sound` says that an accepted program run from an invariant state finishes in a state that satisfies the invariant and records only exclusive writes. `EntryInv` extends `Inv` with the invariant tree, client separation and root completeness, and `checkN_sound_of_entryInv` adds those facts and `Agrees` to the checker's acceptance, concluding the same invariant and fault-free run for the nested program.

The checker's local rules do not name the heap's path structure; the path theory supplies it, composing agreement through take-out, call and put-back and lifting the checker to every finite path. The extraction and restoration stores are not trace events; their target-cell dispositions are the model facts `checkN_extraction_exclusive` and `checkN_extraction_exclusive_of_entryInv`.

4.4 Preservation through the supported fragment

`Supported` is the counted part of the checker's accepted set. A supported program from an accounted state, with the store-slot premise, runs without a fault, records only exclusive writes, satisfies the complete relation at every state it reaches, and satisfies it at the final state at the support the step arithmetic determines.

▷ Claim 5 (Supported execution preserves accounting).

```
theorem supported_run {c : Ctx} {s : Code} (hs : Supported c s)
    {support : List Var} {H : Heap} {env : Env}
    (hacc : Accounted support H env c) (hslots : StoreSlots H env s) :
    ∃ c' H' env' v ws,
      check c s = some c' ∧
      run s env H = .done H' env' v ws ∧
      (∀ w ∈ ws, w.2 = true) ∧
      Accounted (resultSupport s support) H' env' c' ∧
      AccountedAlong H env c support s
```

`AccountedAlong` is the conclusion, never a hypothesis: at each instruction the relation holds at the interpreter's own post-state, and at each array store the target is exclusive. `supported_store_exclusive` states the store's own pre- and post-state: it is reached with its target exclusive and its slot scalar, and the post-state is accounted at the checker's unchanged context. `storeSlots_init` supplies the slot for the initializer's measured write.

Three hard cases.

The projections temporarily raise both counts because the pair's edge and the projected handle are both live; the free of the unique pair removes exactly those two edges, which is why `pair_accounting` returns to the entry counts and the no-free control does not. The suspended parent remains exclusive across the child call because the child's contract is confined to the child's mutable footprint, not because the parent's exclusivity is assumed: `callAt_frame_confined` takes `AnnotationsIn`, checker acceptance, the return variable, the isolated `Inv` and `TakeOutsOK`; the parent/child separation is a `ChildCall.separated` fact used in the path derivation. And the put-back's mutate branch is derived from the protocol boundary and the child-call boundary, not from an assumed final state.

5 Counted protocol and fragment correspondence

5.1 Recognition and the flat correspondence

A source-level discipline is only useful if it survives lowering. The compiler rewrites the program, inlines helpers and chooses the emitted shape; the recogniser walks the actual LCNF value, pinning operation order, arities, operand identities and branch constructors. It rejects the token-preserving near misses (eleven at the top level, eleven on the nested path, fourteen per pair arm), so the correspondence is about the code we name. No universal rejection theorem is claimed. The accepted grammar fixes the check's argument, the array-set operands and their order, the pair constructor and projections, the two `Path` branches and their join, and the continuation boundary.

`project_sound` gives executed/model equality for the abstract prefix; `EntryInv` and layout yield `recognized_write_in_place`, concluding both give `inPlaceOutcome root i v o H`.

`PrefixOutcome` records the target, result/`setResult` binders, heap, disposition, fault and write trace, discarding the variable map and suffix; its trace preserves each write's exclusive/copy bit, and the flat prefix has the single `(root, true)` event. `execAST_recognized` is decoding adequacy, not a semantics; A1–A12 are the independent meanings, which is why decoding is load-bearing.

5.2 The finite nested path

The model nests take-out, call and put-back per level; the finite-path induction discharges the ordinary callee contract from the tail and leaves only the leaf contract and the emitted call's meaning (N10). The flat environment cannot express the emitted per-invocation `oproj[1]` index; that bound is recorded.

`index_path_simulates_model` concludes that the executed nested prefix's `NestedOutcome` is the model run of `Path.build path value body (l :: rest)`, completing with an all-exclusive trace, under the recogniser's verdict, layout, level match, agreement and path contract.

For a finite level path the ordinary child contract is derived at every level: `childContracts_of_path` takes the level list and one leaf computation and yields the ordinary `ChildContract` at each level, whose body is the remaining levels' `build`; `childContract_of_path` is the single-step form. The leaf computation (agreement, checker acceptance, exit variable, count-freedom, annotation) is the one supplied function-level fact.

The counted call is not a consequence of that induction. `CountedChildCall` adds the field `accounted`, the caller's complete accounting at the body's return; every theorem that consumes the counted contract takes that field as an explicit premise, and the closed instance discharges it from the caller's own accounting at the return (`calledAccounted`, `OwnershipTest/Principal.lean`). Only the ordinary `ChildContract` is derived level by level. The claim is the finite path's syntax, not data-dependent recursion. The emitted callee is not captured: `execNested`'s call rule runs the admitted model body with only the call's state, path and value (N10); a deeper level that re-indexes by re-projecting the continuation is outside the model.

5.3 Counted extraction, call and restoration

The take-out builds a pair of the parent and extracted child; the constructor moves the handles in, the projections increment both, and freeing the unique pair releases both edges. The counts return to their entry values only because the pair is freed: the negative controls show that omitting the free, or retaining an alias, leaves a count of two.

▷ Claim 6 (Pair accounting).

```
theorem pair_accounting {H : Heap} {p c : Ref} {i : Nat}
    (hw : WellFormedEntry H p c i) :
    let B := protocol H p c i
    B.2.rc p = 1 ∧ B.2.rc c = 1 ∧ B.2.cells p ≠ none ∧
      B.2.cells c ≠ none ∧ B.2.cells B.1 = none ∧
      Heap.Exclusive B.2 p ∧ Heap.Exclusive B.2 c ∧
      B.2.rc p = H.rc p ∧ B.2.rc c = H.rc c
```

The no-free control, `no_free_components_not_exclusive`, leaves both components at count two, neither exclusive, and the pair live; `WellFormedEntry` fixes both at count one.

Stage	`EnvOwnerCount` (roots)	`IncomingCount` (edges)	parent rc	child rc	disposition / why
entry	1 / 0	0 / 1	1	1	`WellFormedEntry`: root handle; child in the parent slot
take-out	0 / 0	1 / 1	1	1	`takeOut_transfers`: handles moved into the pair
constructor projections	1 / 1	1 / 1	2	2	`incPairComponents_eq`: pair edge plus projected handle
free unique pair	1 / 1	0 / 0	1	1	`pair_accounting`, `pair_traffic_neutral`: edges released
child call	1 / 1	0 / 0	1	1	`CountedChildCall`: parent suspended; child isolated
put-back	1 / 0	0 / 1	1	1	`putBack_exclusive_of_wellFormed`: mutate; child restored to the slot

Table 2 The counted protocol at an accounted caller entry. The pair-live state is outside `Accounted` (checked on the concrete pair-live state, `pairPhase_refused`); intermediate rows are per-stage count effects from the named lemmas, not boundary facts.

`counted_takeOut` consumes the array's cell and selected slot plus the context facts from `checkNF`; it derives the boundary accounting, `HeapCompat`, the pair's freshness and cursor advance. `CountedChildCall` adds one field to the model child-call contract: the caller's `Accounted` at the child's return; it is an explicit premise, not a consequence of the finite-path induction. `counted_restore` derives the mutate branch and accounted post-state; `putBack_exclusive_of_wellFormed` concludes that branch and `Heap.Exclusive` for both the parent and restored child; `counted_nested_chain` relates the counted post-state to `runN` by `HeapCompat`. The pair-live window is not an accounting failure: `incPairComponents_eq` gives the projections' component increments, and `pair_traffic_neutral` gives the release's boundary neutrality—the parent's and child's counts are their entry counts, the child's cell is the entry's, and the temporary pair is gone—so `Accounted` is a relation the protocol attains at its boundaries. The pair-live state is outside it by the count equation: the pair's field contributes an incoming edge to the parent while the caller's variable remains an owning handle for the same reference, so the equation fails there. At the concrete pair-live state of `OwnershipTest/CountedNested.lean` the refusal is kernel-checked (`pairPhase_refused`). This is a boundary relation, not an every-instruction invariant.

The counted interpreter executes five heap-touching operations. The recognised fragment splits at the pair's release: `countedPrefix` is the release-terminated span rather than a numeric cutoff, and `fragment_decomp` records that the rest of the trace (`countedSuffix`) follows it. On that span `execCounted_protocol_boundary` reports the protocol boundary heap, `HeapCompat` to the abstract take-out heap, with its non-heap fields (including the write trace) agreeing with `execNested`'s. From that boundary state `execCounted_full_fragment` runs the rest of the trace: the child assertion, the call at the boundary state, the restoration at the call's output heap, and the return. Its entry point is `execCountedAST`, the counted counterpart of `execNestedAST`, so the recogniser's verdict is spent by the theorem. The counted outcome and the model's run of the same extraction/call/restoration term are related by `NestedOutcome.Compat`: the heaps are `HeapCompat`—the same cell at every name, with the model cursor no higher than the counted one—and the result, the fault and the

write trace are literally equal. `HeapCompat` is still not an invariant in the pair-live window, so the correspondence is stated between the accounted boundary states, not step by step. Extraction and restoration stores are not `Write` records in either interpreter's trace, so the trace does not encode them, although the restoration's execution is a counted step; their target-cell dispositions are the separate model facts `checkN_extraction_exclusive` and `checkN_extraction_exclusive_of_entryInv`. Event representation and target exclusivity are different claims: the second is proved at the model level and the first is not attempted.

The take-out shape is `projectNested`'s `Path.index` arm; the left/right arms are `projectPair`'s, which reads the model term off the operands. `Ownership.Pair.runP` correspondence is a named next step; the protocol is the accounting model, not that correspondence.

5.4 The runtime model

`OwnershipLcnf.RuntimeModel` is a second operational model, related to `Ownership.Heap` by `Corr`: the correspondence is between two models and binds no physical C object or address. It preserves refs, cells, counts, persistence, fields and cursor, so model exclusivity implies the check returns and the write mutates.

Three statements carry the reading: `checkExclusive_no_abort` (under `Corr` and model exclusivity the check never returns `abort`), `setAt_inPlace_of_entryInv` (the set returns in place, preserves `Corr`, keeps the target and writes the scalar), and `prefix_abort_impossible` (the same at the emitted-semantics boundary). None binds a physical C object or address.

"In place" throughout is the modelled disposition: the array operation neither copies nor allocates under the contracts these statements take. It is not a zero-allocation claim for the protocol, whose take-out temporarily allocates the pair. The residual assumptions are the modelled primitives' fidelity to the runtime (`lean_is_exclusive` and `lean_array_uset`), replay-to-backend identity and the identification of a model reference with a runtime object. The linked gate observes the primitives' behaviour on the certified shapes, including whether the write changes the address, and proves none of it; replay-to-backend identity and reference identification are not measured.

5.5 The composed guarantee

The model, the recognised LCNF and the runtime model are separate semantics. Their composition at the flat recognised write is `principal_write`:

▷ Claim 7 (Flat principal guarantee).

```
theorem principal_write
    (hproj : projectPrefix code = some p) (hl : Layout assign p)
    (hEntry : EntryInv H env (Runner.initCtx root) root client)
    (hag : Agrees assign p s env H root i v)
    (hcell : H.cells root = some o) (hin : i < o.fields.size)
    (hc : Corr H R) (ho' : R.cells root = some o') :
    PrefixOutcome.ofExec assign p (execAST assign p code s) =
        some (inPlaceOutcome root i v o H) ∧
      PrefixOutcome.ofModel (Ownership.run (Prefix.toModel assign p) env H) =
        inPlaceOutcome root i v o H ∧
      ∃ H', Runtime.setAt R root i (.scalar v) = some (.inPlace H' root) ∧
        Corr (inPlaceOutcome root i v o H).heap H'
        -- and the target and field conclusions
```

The proof is the conjunction of `recognized_write_in_place` and `Runtime.setAt_inPlace_of_entryInv`; it takes no accounting premise. `Accounted` is consumed at the initializer by `principal_accounted`, which yields `OwnersAccounted`, `CountFidelity` and the count-one dichotomy; `alias_not_accounted` shows the environment side cannot be omitted. Both heaps in the conclusion are models, so the model's count one is not a physical count. The nested companion is one level of a finite supported path.

▷ Claim 8 (The nested counted-fragment statement).

Work at one level of a finite supported path, with array r_A, extracted child c_{ref}, index i, and write B for the second component of `CountedPair.protocol` at the caller's heap, r_A, c_{ref} and i, m' for the call-entry environment (the caller's with the parent bound to r_A and the child to c_{ref}), c' for the call-entry context after the handle move and the child's ownership bind, and H_{out} for the heap the child call leaves.

Premises. (P1) the recogniser accepts the declaration (`projectNested`, yielding p); *(P2)* the layout (`NestedLayout`); *(P3)* the entry machine/model agreement (`NestedAgrees`, for the machine s: an empty entry write trace, the child binder and the call path); *(P4)* the level's data (`PathContract` and `LevelMatch`); *(P5)* the caller's complete accounting (`Accounted`); *(P6)* the counted child call at B, m', c' (`CountedChildCall`). The child call's boundary component is the put-back's `ChildCall`; its `accounted` component is the caller's accounting at the body's return.

Conclusion, for a counted final heap H_f and a model heap and environment. *(a)* The counted interpreter's recognised-code entry point—the counted counterpart of `execNestedAST`, written `execCountedAST`—runs the whole recognised fragment *p.fragment* from the entry machine to a completed state whose heap is H_f, whose result is the caller's array handle, whose restored binder holds that handle, and whose write trace is the entry trace followed by the child's. *(b)* The model run of `Path.build path value body (l :: rest)` completes with that handle and an all-exclusive trace. *(c)* The counted and model outcomes agree by heap compatibility—the same cell at every name, with the model cursor no higher than the counted one—and literally on the result, the fault and the write trace. *(d)* The accounting holds over the extended support—the caller's, with the array handle replaced by the parent and the child's handle added—at the boundary heap B, and over the support the call's result binding and the restoration's renamings determine at the counted final state. *(e)* H_f and the model heap are heap-compatible. *(f)* At B the fresh cursor cell is empty, the cursor advances by one, both r_A and c_{ref} are exclusive, and the boundary heap is heap-compatible with the abstract take-out heap (the caller's heap with the parent's selected slot holding the placeholder scalar). *(g)* At the call-output heap H_{out} the put-back selects `mutate`, writes c_{ref} into the parent's selected slot at the parent object the caller's cell determines (`parentAfter`), and leaves the restoration's heap as H_f. *(h)* If a name lies outside the child's mutable footprint at B, the child call leaves its cell as B has it; and the restoration changes no cell other than the caller's array.

(a) and (c) are ***`principal_counted_fragment`****, whose recognised-code entry point,* ***`execCountedAST`****, runs the whole fragment; the conjuncts (b) and (d)–(h), the recognition-to-model equality* ***`NestedOutcome.ofExec ... = some (NestedOutcome.ofModel ...)`****—and the counted protocol's boundary over the recognised fragment's prefix are the kernel-checked content of* ***`principal_supported_path`****. Both are instantiated closed at the reified declaration (Section 6); premise providers are Table 3.*

`PathContract` is supplied, and `childContracts_of_path` derives the ordinary `ChildContract` at every level; the leaf body's own contract and N10 remain. The counted child call is not derived: `CountedChildCall` is an explicit premise, and `OwnershipTest/`

`Principal.lean` supplies both of its components. Whole-path non-growth outside the root's footprint is not concluded: it needs the leaf's write account.

`Prefix.toModel_supported` puts the certified flat model in the counted fragment, so `supported_run` applies once `StoreSlots` is supplied; `storeSlots_init` supplies that fact only at `execSet`, not for general `p.toModel`. The results are adjacent, not one theorem.

Contract	Provider and status
`EntryInv`, `Agrees`; A1–A12 / N1–N12	client/implementation: unique tree, separation, completeness; agreement; primitive/layout/path/call signatures; `initEntryInv` for fresh arrays; N10 and the supplied leaf remain; not proved of the C runtime
`OwnersAccounted`, `CountFidelity`	model-layer: both are implied by `Accounted` (`ownersAccounted`, `countFidelity` at its support); the checker's take-out/`dec` certificates are carried by `Supported`; `StoreSlots` is supplied; residual runtime fidelity is not proved
`projectNested` verdict, `NestedLayout`, `LevelMatch`, `NestedAgrees`	recognition and layout: the walk at this level, the emitted fresh binders (slots 0–17), the level's operand identification, and the entry machine/model agreement (empty entry write trace, child binder, call path); the instance supplies them as `captured_hproj`, `layout`, `levelMatch` and `agrees`
`PathContract` (level data, leaf)	client: the level list's slot, chain, distinctness, environment and level-context facts, and the leaf's agreement, checker acceptance (`checkBody`), exit variable, count-freedom and annotation; `childContracts_of_path` then derives the ordinary `ChildContract` at every level; the emitted callee's body is not captured
`Accounted` (caller entry, P5)	supplied: the instance's `entryAccounted`; a fresh initializer satisfies it by `Accounted.initCtx`, packaged with the entry invariant as `Entry.initAccounted`; the entry invariant does not imply it (`aliasEntryInv`)
`CountedChildCall` (counted call, N10)	client: the put-back's `ChildCall` boundary plus `accounted`, the caller's complete accounting at the body's return; the finite-path induction does not derive `accounted`; the instance supplies it as `calledAccounted`
level facts (cells, slots, index, multiplicities)	derived at the level from `PathContract`'s `slots`, `envok` and `levelCtx` with `LevelMatch`'s binder identification; `checkNF_build` gives the remaining levels' checker acceptance; no premise added
`Corr`; linked runtime; source correspondence	proved/empirical/assumption: `Corr` is between two models and binds no physical C object or address; the linked gate is evidence, not proof; source correspondence is not attempted

Table 3 Supplied contracts, level-derived facts and model-layer predicates, classified by provider.

6 The closed instance and the evidence

6.1 The instance and its controls

`principal_instance` inhabits `principal_supported_path` at the recogniser's layout on a reified transcription of the captured declaration; every premise is supplied, so the conclusion has no hypotheses. It is a 25-conjunct existential; the first group is below and 22 further conjuncts cover the counted boundary, restoration, counted prefix and locality.

```
theorem principal_instance :
    ∃ Hm envm ws Hf,
```

```
      NestedOutcome.ofExec assign capturedPrefix ... = some (NestedOutcome.ofModel
    ...) ∧
      Nested.runN (Path.build pathVar valueVar body [1]) entryEnv H =
        .done Hm envm (.ref 0) ws ∧ (∀ w ∈ ws, w.2 = true)
      -- and 22 further conjuncts
```

Abbreviated at the first group; the full existential is in `OwnershipTest/Principal.lean`.

`counted_fragment_instance` inhabits `principal_counted_fragment` at the same data. The instance is a closed definition whose type is inferred from the theorem, and its `#print axioms` record is the footprint; a second, concrete re-ascription of the 21-conjunct conclusion is not present in this revision.

The retained-parent controls are in the boundary fixture: `retained_parent_not_mutate` shows a count-two parent does not mutate, and `retained_parent_copies` records the copy and the untouched sibling. A T1-specific `HeapCompat` non-vacuity control is not a declaration in this revision; the copy branch allocates at the entry cursor while the model's unconditional set does not, which is the fact the relation exposes rather than a separate T1 theorem. `OwnershipTest/PathRecursion.lean` is the depth-two witness; `OwnershipTest/PathConsolidation.lean` supplies a depth-three leaf, not an instance. The listed declarations are kernel-checked.

6.2 Evidence

A compiled probe compares the model's predicates with `lean_is_exclusive` and address identity on a corpus including a unique array, a live alias, the alias dropped back to one, a persistent literal and a closure-held array. The closed-pure-array negative lifts a pure closed array to a `___closed` constant read through `lean_obj_once`, so it is shared and copies although the source looks unique. The gate binds the pinned artifacts, IR shapes, C and header; this is evidence, not equivalence.

Case	Checker/model	Runtime disposition	Probe mode	Captured status
unique array	accepted	exclusive, in place	`differential`	exit 0
live alias (count two)	rejected	shared, copy	`differential`	exit 0
closed-pure-array	not certified	shared, copy	`inline-exact`, `closed-case`	exit 0 unchecked; exit 1 armed
nested `setAt`	model/LCNF accepted	model in place; pair allocation	no probe mode	C/IR gate result

Table 4 Observations at revision `d330fa0`, 2 November 2026: flat-probe alignment and nested LCNF/IR gate; probe rows are observations, the nested row a gate result.

"In place" in the table is the modelled disposition defined with the runtime model above, not a zero-allocation claim: the nested row's temporary pair is allocated.

The table records observations at revision `d330fa0` (2 November 2026).

6.3 Theorem map

The table maps declarations and support lemmas to roles; it is not exhaustive.

Declarations	Role
`check_sound`; `checkN_sound_of_entryInv`; `Accounted` (`mult_eq`, `count_one_dichotomy`, `at_most_one_owner`, `countFidelity`); `initCtx`; `initCtx_accounted`; `OwnersAccounted`; `alias_not_accounted`; `aliasEntryInv`; `accountedNotEntryInv`; `Supported`; `supported_check`; `supported_run`; `supported_store_exclusive`; `storeSlots_init`	checker; accounting and independence; preservation with `Accounted` + `StoreSlots`
`project_sound`; `recognized_write_in_place`; `execAST_recognized`; `index_path_simulates_model`; `childContracts_of_path`; `childContract_of_path`	recognition and correspondence
`pair_accounting`; `pair_traffic_neutral`; `boundary_correspondence`; `no_free_components_not_exclusive`; `pairPhase_refused`; `counted_takeOut`; `CountedChildCall`; `counted_restore`; `counted_nested_chain`; `HeapCompat`	counted protocol; N10
`putBack_exclusive_of_wellFormed`; `execCounted_protocol_boundary`; `checkExclusive_no_abort`; `setAt_inPlace_of_entryInv`; `prefix_abort_impossible`; `checkN_extraction_exclusive`; `checkN_extraction_exclusive_of_entryInv`	restoration; counted prefix; runtime model; store targets
`principal_write`; `Prefix.toModel_supported`; `principal_accounted`; `principal_supported_path`; `principal_instance`	flat and nested guarantees

Table 5 Compact theorem map.

7 Related work

Huisinga's 2023 thesis [4] is the closest predecessor. It analyses a model of Lean's IR rather than Lean source or emitted LCNF, enforces uniqueness through a static analysis, and studies the analysis's checker; uniqueness is a static property of the analysis, not the runtime count; there is no compiler link; and soundness and compiler integration remain future work there. This work differs in what is connected: it proves a count-one checker against a model heap and composes that result with the recognised emitted prefix and a runtime model at the flat write, while the counted protocol is isolated at its accounted boundaries. It is not a claim to the first static uniqueness analysis for Lean.

Perceus [9] and Counting Immutable Beans [10] turn ownership into a compilation discipline: an ownership pass inserts counts and reuse, the runtime maintains them, and the correctness argument is for the transformation. This work shares the count-one target but asks a different question: a checker over a model, with a proved relation between owning roots and incoming heap edges, boundary accounting for the emitted extraction pair, and an exclusive restoration derived from the protocol boundary. Perceus optimises allocation and reuse; `Accounted` specifies when count one is justified.

Linear Haskell [2] makes linearity a type-system property through linear arrows and multiplicities; Clean's uniqueness typing [1] is uniqueness, not linearity. Neither is a count-one checker over an existing compiler IR. Lean's `Array.markLinear` [6] is dynamic: it can copy an array to establish uniqueness and, with `LEAN_ABORT_ON_NONLINEAR`, abort on later nonlinear use. Translation validation [8] checks a particular compiler run, not a static discipline. Verified compilation [7, 5] proves a compiler correct for its source language; it is a different category, and we make no comparison row for it.

Table 6 separates what is analysed from what is enforced: type systems and compilers act on their host language, run-time facilities act per run, and this development checks the emitted fragment and proves the connection to a model and a runtime model, with the linked

System	Analysed object	Enforcement	Count representation	Compiler connection	Proof status
Huisinga 2023 [4]	Lean IR model	static analysis	static uniqueness	none	soundness future work
Perceus / Counting Beans [9, 10]	compiler and runtime	ownership pass	runtime count, reuse	compiler and runtime	transformation correctness
Linear Haskell [2]	source language	type checker	linearity, multiplicities	language extension	type-system guarantee
Clean uniqueness [1]	source language	type checker	uniqueness types	language	type-system guarantee
`Array.markLinear` [6]	Lean array API	dynamic mark/copy-/abort	runtime count, flag	runtime facility	runtime test
This work	LCNF fragment and models	external checker	roots and edges	model and runtime-model link	kernel-checked

Table 6 Related guarantees, analysed at their own boundary.

C runtime a measured assumption.

8 Limits and conclusion

▷ Non-claim 1. The C semantics of the primitives, replay-to-backend identity, address identity, free and cascade, and the source correspondence are not formalised or attempted; the emitted recursive callee and continuation (N10) are not captured; the flat environment cannot express the per-invocation index; the leaf body and several heap-shape facts are supplied; persistence, concurrency and callbacks are outside the fragment; and the nested linked runtime is not measured.

The accounting layer is not a consequence of `Inv`: `OwnersAccounted` has a kernel countermodel and explicit `bindOwned`/`dec` preconditions, and the checker's certificates are carried by `Supported`. `CountFidelity` is the count equation carried by `Accounted`, but the residual runtime fidelity is not proved. `StoreSlots` is supplied, and `storeSlots_init` covers only the initializer's measured write; the fragment excludes `.inc` and admits only `.dec x 1`.

The composed statements do not carry accounting across models. `principal_write` takes no accounting premise and does not conclude one; `principal_accounted` consumes `Accounted` at the initializer. `principal_supported_path` takes the caller's complete accounting (`hacc`) and the counted child-call contract (`hcall`) and concludes `Accounted` at the counted boundary and at the restored state, with `HeapCompat` between the counted and model heaps; no statement carries either relation to a physical counted reference. Its child call is an admitted model body, not the emitted callee, and its conclusion is the reached in-place store at one level, not whole-path non-growth outside the root's mutable footprint. The counted interpreter's own trace is related to the model's run over the whole fragment (‘execCounted_full_fragment‘, consumed at the level by ‘principal_counted_fragment‘), but the relation is ‘HeapCompat‘ plus equality on the result, the fault and the write trace: the same cell at every name, with the model cursor no higher than the counted one, ‘HeapCompat‘ is not an invariant inside the pair-live window, and no step-by-step agreement between the two interpreters is claimed. The nested trace omits extraction and restoration stores—neither interpreter records them as `Write` records—but each target cell is proved exclusive by `checkN_extraction_exclusive_of_entryInv`, a fact about the store's target

rather than a trace event, and not a silent absence. The pair-arms correspondence with `Ownership.Pair.runP` is a named next step.

The recognition claim is scoped to the accepted grammar and the mutation corpus, not a universal rejection theorem. The differential mode compares recorded expectations rather than executing the `Ownership` model. The probe returns two on an ordinary mismatch, but `sealed` returns one and so cannot be distinguished from the exclusivity abort; that path is not exercised. The table's observations are at revision `d330fa0` (2 November 2026).

Conclusion.

The count-one discipline is proved at the model, the recognised LCNF and the runtime model for the flat write, composed as `principal_write`, which takes no accounting premise. The nested `principal_supported_path` composes the finite-path correspondence, the counted chain and the counted-prefix boundary at one level, from an accounted caller state (`hacc`) and conditional on the child-call contract (`hcall`); the counted interpreter's run over the whole *p.fragment* (`principal_counted_fragment`) is `NestedOutcome.Compat` to the model's run: heap compatibility with the same result, fault and write trace. Neither a step-by-step agreement inside the pair-live window nor the emitted callee is claimed. The result buys a checked count one for a recognised fragment, not a type system or a physical count. Next: the C semantics, replay/backend and address identity, free and cascade, and the emitted callee; soundness is proved, not completeness.

Acknowledgements

AI assistance.

The development and this manuscript were produced with substantial AI assistance. Tools and model versions used across the development were not recorded; the model identifier recorded for this manuscript revision is `deepseek/deepseek-flash`. Categories: prose, Lean proofs, code, analysis, literature search. The division of labour between the author and these tools is not recorded; the tools produced the section drafts and theorem statements here and assisted the Lean modules and scripts.

Outputs were checked by the pinned kernel build and axiom audit; a declaration census checks every displayed name against the development, and the statements and claims are checked against the development's declarations. Kernel checking is not human validation: proofs were machine-generated and spot-checked, not checked line by line. Dagstuhl's statement (checked 2 November 2026) requires disclosure beyond proofreading; the conference and publisher policy must be rechecked at submission. Paraphrasing does not remove substantive AI involvement.

References

1 Erik Barendsen and Sjaak Smetsers. Uniqueness typing for functional languages with graph rewriting semantics. *Mathematical Structures in Computer Science*, 6(6):579–612, 1996.

2 Jean-Philippe Bernardy, Mathieu Boespflug, Ryan R. Newton, Simon Peyton Jones, and Arnaud Spiwack. Linear haskell: Practical linearity in a higher-order polymorphic language. In *Principles of Programming Languages (POPL)*, 2018.

3 Leonardo de Moura and Sebastian Ullrich. The lean 4 theorem prover and programming language. In *Automated Deduction – CADE-28*, volume 12699 of *LNCS*. Springer, 2021.

4 Marc Huisinga. Static uniqueness analysis for the lean 4 theorem prover. Master's thesis, Karlsruhe Institute of Technology, 2023. URL: `https://pp.ipd.kit.edu/uploads/publikationen/huisinga23masterarbeit.pdf`.

5 Ramana Kumar, Magnus O. Myreen, Michael Norrish, and Scott Owens. Cakeml: A verified implementation of ml. In *Principles of Programming Languages (POPL)*, 2014.

6 Lean FRO. Array.marklinear, 2025. Lean API documentation. URL: `https://lean-lang.org/doc/api/Init/Data/Array/Basic.html#Array.markLinear`.

7 Xavier Leroy. Formal verification of a realistic compiler. *Communications of the ACM*, 52(7), 2009.

8 George C. Necula. Translation validation for an optimizing compiler. In *Programming Language Design and Implementation (PLDI)*, 2000.

9 Alex Reinking, Ningning Xie, Leonardo de Moura, and Daan Leijen. Perceus: Garbage free reference counting with reuse. In *Programming Language Design and Implementation (PLDI)*, 2021.

10 Sebastian Ullrich and Leonardo de Moura. Counting immutable beans: Reference counting optimized for purely functional programming. In *Implementation and Application of Functional Languages (IFL)*, 2019.